
UNIS

HAREKETLİ UYDU TERMINALI

Proje Kapsamli Teknik Sunum

BTK Satellite Terminal Yarismasi

Hazirlanma Tarihi: 15.05.2026

Yazilim Ekibi: Kerim, Efe, Elif, Simge

ROS 2 Humble | Gazebo Classic | C++ | Python | Tkinter

Icerik

Sunum Akisi

#	Baslik	Aciklama
1	Proje Genel Bakis	Proje tanimi, amac, kapsam
2	Sistem Mimarisi	4 katmanli mimari ve veri akisi
3	Bilesen 1: Gomulu Yazilim	Kerim - 5 adimli STM32/Teensy C++ kodu
4	Bilesen 2: ROS 2 & Gazebo	Efe - Kontrol node'lari, simulasyon, dataset
5	Bilesen 3: Python Arayuz	Ekip - Tkinter GUI, uydu hesabi
6	Bilesen 4: Stabilizasyon	Simge/Elif - C++ referans kodu
7	Sartname Karsilama	13 gereksinim analizi
8	Performans & Metrikler	Hiz, dogruluk, CPU kullanimi
9	Yol Haritasi	KTR hazirlik ve ilerleme plani
10	Sonuc	Genel degerlendirme ve cikarimlar

BOLUM 1

Hedef, Amac ve Bilesenler

Proje Tanımı

Hareketli Uydu Terminali (SOTM)

UniS projesi, BTK Satellite Terminal Yarismasi icin gelistirilen bir Hareketli Uydu Terminali (Satellite On The Move - SOTM) yazilim sistemidir.

Temel Hedef:

Hareketli bir platform (arac, gemi, Insansiz Kara Araci) uzerindeki antenin, platform hareket halindeyken bir haberlesme uydusuna kilitli kalmasini saglamak

Karsilanan Temel Zorluklar:

Platformun ani sarsintilari ve egimlerine karsi gercek zamanli stabilizasyon

Uydunun goreceli konumunun surekli ve hassas sekilde takibi

Sensor gurultusu ve mekanik titresimlerin filtrelenmesi

Hedef kaybi durumunda hizli yeniden kilitleme (8 sn)

Insan makine arayuzu uzerinden kolay kontrol

Proje Bilesenleri

4 ana bilesen, 4 ekip uyesi

Bilesen	Kisi	Dil	Detay	RoI
Gomulu Yazilim	Kerim	C++	STM32/Teensy 5 adimli kontrol dongusu EKF, SGP4, PID, Telemetry	Donanim uzerinde gercek zamanli kontrol
ROS 2 & Gazebo	Efe	C++ Python	ROS 2 Humble node'lari Gazebo Classic simulasyon Dataset dogrulama araci	Algoritma dogrulama ve simulasyon
Python Arayuz	Ekip	Python Tkinter	Kullanici arayuzu Uydu yonelim hesabi ROS bridge	Insan-makine arayuzu
Stabilizasyon Ornegi	Simge Elif	C++	PID + low-pass filtre Referans implementasyon	Referans kod ve ogrenme

Teknik Yigin

Kullanılan teknolojiler ve araclar

Alan	Kullanilan Teknoloji
Platform	STM32 (HAL), Teensy, FreeRTOS, Bare-metal C++
ROS Sürümü	ROS 2 Humble (Ubuntu 22.04)
Simulasyon	Gazebo Classic 11 + ros2_control
Arayuz	Python 3 + Tkinter + reportlab
Gelistirme	CMake, colcon, VS Code, Git
Sensorler	IMU (MPU-6050), GPS (NMEA), QPD, Encoder, Limit Switch
iletisim	UART (115200 baud), ROS 2 Topics/Services
Gorev Yonetimi	FreeRTOS (opsiyonel), bare-metal super-loop

BOLUM 2

4 Katmanli Veri Akisi ve Kontrol Mantigi

Mimariye Genel Bakis

4 katmanli moduler yapı

Sistem yazilimi, her biri belirli bir sorumluluga sahip 4 ana katmandan olusur. Veri akisi asagidan yukariya dogru ilerler ve her katman bir oncekinin ciktisini kullanir.

Katman	Bilesenler	Frekans	Gorev
1. ALGILAMA (Perception)	IMU (ivmeolcer + jiroskop) GPS (NMEA) Encoder QPD/Lazer Limit Switch	100 Hz 1 Hz 100 Hz Kesikli Her dongu	Ham sensor verilerinin toplanmasi ve on islenmesi
2. KESTIRIM (Estimation)	Kalman Filtresi (EKF) Quaternion Yonelim SGP4 Propagator Koordinat Donusum	100 Hz 100 Hz 1 Hz 1 Hz	Sensor fuzyonu ile anlamli durum bilgisi uretimi
3. KONTROL (Control)	Azimuth PID Elevasyon PID Stabilizasyon PID QPD Aktif Takip Spiral Arama	100 Hz 100 Hz 100 Hz 100 Hz Olay bazli	Motor komutlarinin hesaplanmasi ve uygulanmasi
4. ARAYUZ (Interface)	Tkinter GUI ROS 2 Bridge Telemetry Parametre Yonetimi	Kullanici etkilesimi	Insan-makine arayuzu ve sistem yonetimi

Veri Akisi Semasi

Katmanlar arasi veri akisi

```
IMU/Gyro
| (100 Hz, ham accel + gyro)
v
+-----+
| EKF (7-durumlu) | -> Quaternion -> Roll/Pitch/Yaw
| Kalman Filtresi | -> Gyro bias tahmini
+-----+
|
v
+-----+
| STABILIZASYON PID | -> X/Y Itki Motor Komutu
| (hedef roll=0, |
| hedef pitch=0) |
+-----+

GPS + Uydu Parametresi
| (1 Hz, LLA + TLE)
v
+-----+
| SGP4 Propagator | -> Uydu ECI Konumu
| + Koordinat Donusum | -> Az/El Acilari
+-----+
|
+-- Platform Tilt Telafisi (roll/pitch ile duzeltme)
v
```

```
+-----+
| AZIMUTH/ELEVASYON | -> Az/El Motor Komutu
| PID Kontrol |
+-----+

QPD/Lazer
| (hataya gore)
v
+-----+
| QPD Hata Vekturu | -> Az/El Offset Duzeltmesi
| ex = (A+D)-(B+C)/T |
| ey = (A+B)-(C+D)/T |
+-----+

Encoder -> PID Geri Beslemesi (kapali dongu)
GUI -> Mod/Hedef/Parametre Komutlari
```

Kontrol State Machine

Sistemin calisma modlari

Sistem, asagidaki durum makinasi ile yönetilir:

Durum	Aciklama	Detay
POWER_ON	Guc acilisi	Donanim baslatma baslangici
HW_SELF_TEST	Donanim oz-testi	IMU, GPS, motor, encoder testi
HOMING	Sifir noktasi	Limit switch -> encoder=0
CALIBRATING	IMU kalibrasyon	500 ornek bias tahmini
IDLE	Bekleme	Kullanici komutu bekliyor
AUTO_TRACKING	Otomatik takip	EKF + SGP4 + PID + QPD aktif
MANUAL	Manuel mod	Kullanici Az/El girisleri
ERROR	Hata	Motorlar durduruldu, bekleniyor

Gecisler: AUTO <-> MANUAL (kullanici komutu), herhangi bir durumdan ERROR (hata tespiti)

BOLUM 3

Kerim — 5 Adimli Kontrol Dongusu (C++)

5 Adimli Mimari

STM32/Teensy uzerinde gercek zamanli kontrol

Gomulu yazilim, bir mikrodenetleyici (STM32 veya Teensy) uzerinde calisan, 5 adimdan olusan bir kontrol dongusuden olusur. FreeRTOS veya bare-metal ortamda calisabilir.

Ad im	Islev	Dosya	Detay	Periyot
1	BASLATM A VE KALI BRASYON	initialization.h	Donanim testi Homing IMU kalibrasyonu	1 kez (sistem acilista)
2	VERI TOPLAMA	main.cpp	IMU 100Hz GPS 1Hz Encoder 100Hz UART komut	Her 10ms
3	SENSOR FUZYONU & KOORDI NAT	SensorFusion.h SatelliteTracker.h	EKF predict+update SGP4 propagasyon Platform tilt telafisi	Her 10ms
4	PID KONTROL & GUVEN LIK	PIDController.h	Az/EI PID Anti-windup Deadband SafetyMonitor	Her 10ms
5	TELEMET RI & GERI BESLEME	telemetry.h	72 byte UART paketi CRC16 Komut kuyrugu	Her 100ms

Toplam Dongu Performansi: ~3.3ms (hedef 10ms) -> %67 CPU Kullanimi

Adim 1: Baslatma ve Kalibrasyon

Detayli anlatim

1A — Donanim Oz-Testi (HardwareSelfTest):

Sistem enerji aldigi anda tum donanim bileşenlerini test eder:

IMU: 0x68 adresinde MPU-6050 algılanması ve yer çekimi ölçümünün kontrolü ($9.80665 \pm 2 \text{ m/s}^2$)

GPS: NMEA modulu ile UART iletişimi

Motor sürücüler: Azimut ve elevasyon motorları

Encoder'lar: Her iki eksen açısı sensörü

Herhangi bir test başarısız olursa ilgili hata bayrağı set edilir ve sistem ERROR moduna alınır.

1B — Homing (HomingProcedure):

Antenin referans (sifir) noktasının belirlenmesi:

1. CW yönünde limit switch'e doğru yaklaşma (%5 PWM)
2. Limit switch tetiklenince anında durma
3. 100ms bekleme (mekanik salınım sonlansın)
4. CCW yönünde 2 derece geri çekilme (%3 PWM, 400ms)
5. Bu nokta encoder=0 olarak belirlenir

15 saniye timeout: limit switch bulunamazsa hata.

1C — IMU Kalibrasyonu (IMUCalibration):

Cihaz hareketsizken 500 örnek (5 saniye) toplanır:

Gyro Bias: Hareketliken ortalama gyro değeri (ideal: 0). Bu değer EKF'ye başlangıç değeri olarak verilir.

Accel Bias: normalized yer çekimi vektöründen sapma.

Kalite Kontrol: Accel scale %5 tolerans, gyro norm < 0.1 rad/s

Adim 2: Veri Toplama

Sensor okuma ve on isleme

Ana kontrol dongusunun her adiminda (10ms) asagidaki islemler gerceklestirilir:

Sensor	Frekans	Veri	Kullanım
IMU (MPU-6050)	100 Hz (10ms)	3-eksen ivmeölçer (m/s ²) 3-eksen jiroskop (rad/s)	Kalibrasyon bias'i çıkarılır EKF predict+update girdisi
GPS (NMEA)	1 Hz (1000ms)	Enlem/Boylam/Irtifa HDOP, hız, yön Uydu sayısı	HDOP < 5m ise geçerli fix yoksa hata bayrağı
Encoder	Her dongu (10ms)	Azimut açısı Elevasyon açısı Anlık hız (dps)	PID geri beslemesi
Limit Switch	Her dongu	Azimut min/max Elevasyon min/max	Güvenlik monitörü girdisi
UART Komut	Olay bazlı	CommandPacket (8 elemanlı FIFO)	Kullanıcı komut kuyruğu

Adim 3A: Extended Kalman Filter

7-durumlu IMU Sensor Fuzyonu

EKF, IMU'nun gyro ve ivmeölcer verilerini birleştirerek terminalin uzaysal yönelimini (orientation) kestirir.

Durum Vektörü (7 boyut):

```
x = [q0, q1, q2, q3, bx, by, bz]  
Birim quaternion Gyro bias (rad/s)
```

EKF İşleyisi:

Predict (Tahmin): Jiroskop ölçümünden quaternion kinematigi ile durum ilerletilir.

```
q_dot = 0.5 * Omega(w_corrected) * q  
w_corrected = w_measured - bias  
7x7 Jacobian matrisi ile kovaryans yayilimi
```

Update (Düzeltilme): İvmeölcer ölçümü ile düzeltilme.

```
h(x) = R(q) * [0, 0, g]^T (beklenen ivme)  
Ölçüm yeniligi: innov = accel_raw - h(x)  
Kalman kazancı: K = P*H'*(H*P*H' + R)^-1  
Güncelleme: x += K * innov, P = (I-K*H)*P
```

İvme büyüklüğü yer çekiminden %30'dan fazla saparsa (100% hareket) update adımı atlanır.

Bu sayede yüksek ivmeli manevralarda hatalı güncelleme engellenir.

Adim 3B: SGP4 Uydu Propagatoru

TLE'den uydu konumuna

SGP4 (Simplified General Perturbations #4), TLE (Two-Line Element) verisini kullanarak yapay uyduların yörünge konumunu hesaplar.

Veri Akisi:

```
TLE (ham metin) -> TLEParser::parse()  
-> TLEData (yapısal veri)  
-> SGP4Propagator::init()  
-> SGP4Elements (önceden hesaplanmış sabitler)  
-> SGP4Propagator::propagate(JulianDate)  
-> ECI konum ve hız vektörleri (km, km/s)  
-> CoordinateTransform::eciToECEF()  
-> ECEF (Dünya merkezli, Dünya sabit)  
-> CoordinateTransform::calcAzEl(gözlemci_LLA)  
-> Az/El açıları (derece) + menzil (km)
```

SGP4 Özellikleri:

- Newton-Raphson ile Kepler denklemi çözümü (8 iterasyon)
- J2, J3 pertürbasyon etkileri dahil
- Atmosferik sürüklenme (BSTAR) modellenmesi
- Perigee < 156 km için özel durum (düşük yörünge)
- Epoch'dan itibaren herhangi bir JD'de konum hesaplama

Koordinat Donusumleri:

ECI -> ECEF: Greenwich Sidereal Time (GAST) ile dünya dönüşü

ECEF -> LLA: WGS-84 elipsoid, Bowring iterasyonu

LLA -> Az/El: NED (North-East-Down) çerçevesinde

Adim 3C: Platform Tilt Telafisi

Arac egiminin matematiksel düzeltmesi

Arac bir yokus veya egimde duruyorsa, uyduya doğru hesaplanan Az/El açıları gövde çerçevesinde geçersiz olur. Bu nedenle EKF'den gelen roll/pitch bilgisi kullanılarak hedef açıları düzeltilir.

Matematiksel Model:

1. Uydu birim vektörü küresel koordinatlardan NED çerçevesine çevrilir.

$$u_x = \cos(e_l) * \sin(a_z)$$

$$u_y = \cos(e_l) * \cos(a_z)$$

$$u_z = \sin(e_l)$$

2. Platform dönüş matrisi uygulanır: $R = R_y(\text{pitch}) * R_x(\text{roll})$

$$b_x = c_p * u_x + s_p * s_r * u_y + s_p * c_r * u_z$$

$$b_y = c_r * u_y - s_r * u_z$$

$$b_z = -s_p * u_x + c_p * s_r * u_y + c_p * c_r * u_z$$

3. Gövde çerçevesinden tekrar Az/El'e:

$$e_{l_comp} = \arcsin(b_z)$$

$$a_{z_comp} = \arctan2(b_x, b_y)$$

Bu düzeltme olmadan, eğimli bir yüzeyde anten yanlış yöne doğrultulmuş olur.

Adim 4: PID Kontrol ve Guvenlik

Detayli PID analizi

PID kontrol, iki bagimsiz eksen icin ayri PIDController siniflariyla gerceklestirilir.

PID Parametreleri:

Parametre	Azimut	Elevasyon	Anlami
Kp (Oransal)	2.5	3.0	Buyuk -> hizli tepki, salinim riski
Ki (Integral)	0.1	0.08	Kalici hatayi sifirlar
Kd (Turev)	0.3	0.25	Salinimi azaltir, yumusatir
Max Integral	20.0	15.0	Anti-windup limiti
Deadband	+/-0.1 derece	+/-0.1 derece	Altinda motor durur
Turev Filtre	0.7	0.7	EMA katsayisi
Max Hiz	60 derece/s	30 derece/s	Guvenlik limiti

PID Formulu:

```
error = target - measured
P = Kp * error
I += Ki * error * dt (anti-windup ile)
D = Kd * (error - prev_error) / dt (filtreli)
output = clamp(P + I + D, min, max)
```

Anti-Windup Mekanizmasi:

Cikis doyuktaysa VE hata ayni yonde ise integratör dondurulur. Bu sayede integral sarmasi (windup) engellenir ve toparlanma suresi kisalir.

Deadband:

+/-0.1 derece icinde hata sifir kabul edilir, integrator sifirlanir ve motor durdurulur. Bu, surekli mikro-duzeltmeleri engelleyerek motor omrunu uzatir ve guc tuketimini azaltir.

Adim 4: Guvenlik Monitoru

4 seviyeli koruma sistemi

SafetyMonitor, fiziksel ve yazilimsal limitleri denetleyerek sistemi guvenli bolgede tutar:

Seviye	Durum	Aksiyon	Renk
SAFE	Guvenli bolge (Normal calisma)	Komut oldugu gibi iletilir Hiz sinirlamasi yok	Yesil
SOFT	Yazilim limiti yakini (Uyari bolgesi)	Hiz %50 azaltilir Kablo koruma aktif	Sari
HARD	Fiziksel limit (Kritik bolge)	Motor DURDURULUR Hata bayragi set edilir	Turun cu
FAULT	Limit switch tetiklendi (Acil durum)	Acil durdurma Sistem ERROR moduna alinir	Kirmizi

Kablo Sarmalama Korumasi:

Azimut eksenini 350 dereceyi gectiginde (kablo sarmasi riski), motor hizi kademeli olarak azaltilir:

```
if (pos >= cable_wrap):  
    scale = 1 - (pos - cable_wrap) / (hard_max - cable_wrap)  
    cmd *= clamp(scale, 0.0, 1.0)
```

Adim 5: Telemetry Paketi

72 byte, UART 115200 baud, 10 Hz

Telemetry sistemi, sistemin anlik durumunu 10 Hz frekansinda PC/GUI'ye bildirir.

Paket Yapisi:

Kategori	Alan	Boyut	Aciklama
Baslik	Magic (0xABCD)	2 byte	Paket baslangici
	Timestamp	4 byte	ms cinsinden sure
Durum	System State	1 byte	8 durum
	Error Flags	2 byte	8 hata kodu
Konum	Latitude, Longitude, Altitude	12 byte	WGS-84
Yonelim	Roll, Pitch, Yaw	12 byte	EKF ciklisi
Anten	Az/El gercek (encoder)	8 byte	Gercek konum
	Az/El hedef (setpoint)	8 byte	Hedef konum
	Az/El hata	8 byte	Takip hatasi
Uydu	Sat Az/El/Menzil	12 byte	SGP4 ciklisi
Sinyal	RSSI	1 byte	dBm
Dogrulama	CRC-16 (CCITT)	2 byte	X-25 polinom

Kategori	Alan	Boyut	Aciklama
TOPLAM		72 byte	

Komut Seti (GUI -> Terminal):

Komut	Kod	Aciklama
SET_AUTO_MODE	1	Otomatik takip modu
SET_MANUAL	2	Manuel kontrol modu
MANUAL_AZ	3	Azimet manuel hedef (float)
MANUAL_EL	4	Elevasyon manuel hedef (float)
LOAD_TLE	5	Yeni TLE yukle
HOME	6	Homing (sifirlama)
EMERGENCY_STOP	7	Acil durdurma

BOLUM 4

Efe — Kontrol Node'lari, Simulasyon, Dataset

ROS 2 Mimarisi

mergen_control paketi detayli inceleme

ROS 2 Humble uzerinde calisan kontrol node'lari, ozel mesaj tipleri ve Gazebo entegrasyonu.

Topic Yapisi:

Topic	Mesaj Tipi	Tip	Aciklama
/mergen/imu_filtered	ImuFiltered	Subscription	Filtrelenmis IMU (roll/pitch)
/mergen/motor_state	MotorState	Subscription	Encoder motor konumu
/mergen/target_angles	TargetAngles	Subscription	Hedef Az/El acilari
/mergen/motor_command	MotorCommand	Publisher	PWM + stabilizasyon ciklisi

Kontrol Node'u (control_node.cpp):

100 Hz timer ile calisan ana kontrol node'u:

3 subscription: IMU filtrelenmis, motor durumu, hedef acilari

1 publisher: Motor komutlari (PWM + stabilizasyon)

Her 10ms'de: Azimuth PID, Elevasyon PID, Stabilizasyon

Cikti: azimuth_pwm, elevation_pwm, stabilizer_x/y_pwm

PID Kontroller (C++)

mergen_control kutuphanesi

mergen_control kutuphanesi 4 ana sinif icerir. Her biri belirli bir kontrol gorevini ustlenir.

1. PidController:

```
PidController az_pid(2.0, 0.02, 0.15, -1.0, 1.0);  
PidController el_pid(2.2, 0.02, 0.16, -1.0, 1.0);  
double output = pid.update(target, measurement, dt);
```

Genel amaclı PID kontrolör. Kp, Ki, Kd parametreleri ve çıkış sınırlaması (clamp) ile yapılandırılır. ROS 2 topic'lerinden gelen verilerle çalışır.

2. StabilizationController:

```
StabilizationController stabilizer(roll_pid, pitch_pid, 4.0);  
auto cmd = stabilizer.update(roll_deg, pitch_deg, dt);
```

EKF'den gelen roll ve pitch açılarını alır, hedef değeri sıfır (düz pozisyon) olan PID'lerden geçirir ve mm_per_degree ölçeğiyle X/Y itki mekanizması komutuna dönüştürür.

3. QpdTracker:

```
QpdError err = tracker.calculate(A, B, C, D);
```

QPD sensorundan gelen 4 bölge akımı (A, B, C, D) kullanılarak boresight hata vektörü hesaplanır.

4. SpiralSearch:

```
SpiralOffset off = search.update(elapsed_sec);
```

Hedef kaybı durumunda giderek genişleyen spiral deseninde arama offseti üretir.

QPD Lazer Takip Algoritması

Aktif geri besleme ile yüksek hassasiyet

QPD (Quadrant Photo Diode), lazer isiginin sensore dusus noktasini 4 bolgeye ayirarak algilar.

Hata Vektörü Hesabi:

```
toplam = A + B + C + D
```

```
error_x = ((A + D) - (B + C)) / toplam
```

```
error_y = ((A + B) - (C + D)) / toplam
```

Bu hata vektörü, antenin boresight (optik eksen) hatasını gösterir.

Çalışma Mantığı:

toplam < detection_threshold: Lazer hedefte değil -> Spiral arama başlatılır

toplam >= threshold & error küçük: Aktif takip -> Hata vektörü Az/El offset'i olarak PID'ye eklenir

error_x -> azimut hedefinde düzeltme

error_y -> elevasyon hedefinde düzeltme

Neden QPD + Lazer?

Teorik uydu yönelimindeki mekanik toleransları kapatır

Platform sarsıntılarına karşı ekstra güvence sağlar

Encoder geri beslemesinden bağımsız çalışır

Milimetrik hassasiyette yönlendirme mümkün

Spiral Arama Algoritması

Hedef kaybi durumunda akilli arama

Teorik konumlandırma sonrası lazer hedefi yakalayamazsa (QPD toplam sinyali esik altı), sistem spiral arama moduna geçer.

Matematiksel Model:

```
radius(t) = min(max_radius_deg, radial_speed * t)
angle(t) = angular_speed * t
offset_az = radius(t) * cos(angle(t))
offset_el = radius(t) * sin(angle(t))
```

Parametreler:

Parametre	Anlami	Birim
radial_speed_deg_s	Yari capin buyume hizi	Derece/saniye
angular_speed_rad_s	Acisal hiz	Radyan/saniye
max_radius_deg	Maksimum arama yaricapi	Derece

Durum Makinasi:

```
QPD toplam < threshold -> SPIRAL_ARAMA
QPD toplam >= threshold -> AKTIF_TAKIP
Hata vector < esik -> KILITLI (locked)
```

Gazebo Simulasyonu

Dijital ikiz ile dogrulama

Gazebo Classic 11, ROS 2 Humble ile birlikte kullanilir. CAD modeli tamamlanana kadar basit geometrilerle simulasyon yapilir.

World Dosyasi (mergen_test.world):

```
Hedef: 3.464m x, 0m y, 2.0m z -> 30 derece elevasyon  
Menzil: 4 metre  
Hedef geometrisi: Kirmizi silindir (radius 0.5m)
```

Controller Yapilandirmasi:

```
controller_manager:  
  update_rate: 100  
joint_state_broadcaster:  
  type: joint_state_broadcaster/JointStateBroadcaster
```

Baslatma (Launch):

```
simulation.launch.py:  
1. mergen_gazebo/gazebo.launch.py  
2. mergen_estimation/imu_filter.launch.py  
3. mergen_control/control.launch.py
```

Sistem Modlari:

```
manual -> Kullanici manuel Az/El giris  
automatic -> EKF + SGP4 + PID + QPD (tum sistem)  
search -> Spiral arama aktif  
safe -> Guvenli mod, motorlar durduruldu
```

Dataset Dogrulama Araci

CAD olmadan algoritma dogrulama

CAD modeli hazir olmadan once kontrol algoritmalarini test etmek icin CSV dataset tabanli bir dogrulama araci gelistirilmistir.

Veri Uretimi (generate_dataset):

```
Sure: 5 dakika (300 saniye)
Frekans: 100 Hz -> 30.000 satir
Roll: 8 * sin(2*pi*t/10) + Gaussian(0, 0.18)
Pitch: 8 * cos(2*pi*t/10) + Gaussian(0, 0.18)
Yaw: (36*t) %% 360 (sabit donus 36 derece/s)
Target: Az=120, El=30 (sabit)
QPD: ex=0.15*sin(0.7*t), ey=0.12*cos(0.5*t)
```

Isleme (run_dataset):

1. Kalman filtresi (Q=0.01, R=0.25) -> IMU filtreleme
2. QPD hata vektörü -> Az/El offset
3. Azimuth PID + Elevasyon PID -> PWM
4. Stabilizasyon PID -> X/Y PWM
5. Kilit durumu: $|ex| < 0.03$ & $|ey| < 0.03$

Kullanım:

```
python dataset_simulator.py --generate --output sample_motion.csv
python dataset_simulator.py --input sample_motion.csv --output sample_result.csv
```

Dataset Cikti Analizi

Sample cikti sutunlari ve anlami

Sutun	Aciklama	Ornek Deger
time_s	Zaman damgasi	3.000
roll_filtered_deg	Kalman filtreli roll	7.59491
pitch_filtered_deg	Kalman filtreli pitch	-2.29486
qpd_error_x	QPD X hata	0.12948
qpd_error_y	QPD Y hata	0.00849
azimuth_pwm	Azimet motor PWM	1.00000 (limit)
elevation_pwm	Elevasyon motor PWM	-0.99320
stabilizer_x_pwm	Stabilizasyon X eksen	-1.00000 (limit)
stabilizer_y_pwm	Stabilizasyon Y eksen	1.00000 (limit)
target_locked	Kilit durumu	False

Gozlemler:

PID ciktisi genellikle limitte (1.0/-1.0) -> yuksek hata

target_locked=False -> QPD hatasi threshold'un ustunde

Kalman filtresi gurultuyu basariyla bastiriyor

Stabilizasyon X/Y surekli olarak roll/pitch'i bastirmaya calisiyor

Firmware Referans Kodu

ROS 2 -> Gomulu platform gecisi

firmware_reference/teensy_stm32_cpp klasoru, ROS 2 kontrol algoritmalarının gomulu platforma tasinmasi için bir referans saglar.

main_loop_example.cpp:

```
MotorOutput update_control_loop(
float target_az, float target_el,
float measured_az, float measured_el,
float raw_roll, float raw_pitch,
float dt_sec) {
static KalmanFilter1D roll_filter(0.01, 0.25);
static KalmanFilter1D pitch_filter(0.01, 0.25);
static PidController az_pid(2.0, 0.02, 0.15, -1, 1);
...
}
```

KalmanFilter1D:

```
class KalmanFilter1D {
float update(float measurement) {
p += q; // covariance + process noise
float k = p / (p + r); // Kalman gain
x += k * (measurement - x); // estimate update
p *= (1 - k); // covariance update
return x;
}
```

```
};
```

1 boyutlu Kalman filtresi, 7-durumlu EKF'nin basitleştirilmiş versiyonudur. Gerçek sistemde EKF kullanılır.

BOLUM 5

Ekip — Tkinter GUI, Uydu Hesabi, ROS Bridge

Tkinter GUI

Kullanici arayuzu detayli analiz

Python Tkinter kutuphanesi ile hazirlanmis 820x560 pencere boyutunda kullanici arayuzu. ROS 2 kurulu olmayan bilgisayarlarda simulasyon modunda calisir.

Ana Bolumler:

Bolum	Bilesenler	Islev
1. Komut ve Mod (ttk.LabelFrame)	RadioButton: Otomatik/Manuel Entry: Azimuth, Elevasyon Button: Manuel Hedef Gonder	Mod secimi Manuel aci giris Hedef iletimi
2. Uydu Yonelimi (ttk.LabelFrame)	Entry: Enlem, Boylam Combobox: Turksat 4B/5A Button: Az/El Hesapla	GPS koordinatlari Uydu secimi Teorik hesaplama
3. Telemetry (ttk.LabelFrame)	Text widget (12 satir) Otomatik kaydirma	Durum mesajlari Sistem geri bildirimi
4. Alt Butonlar	Parametreleri Kaydet Guvenli Mod	JSON config Acil durum

Ozellikler:

ui_state.json ile kalici durum kaydi (mode, acilar, konum)

Elevasyon girisi 0-90 derece sinirlamasi

Otomatik modda uydu konumuna gore Az/El hesaplama

Manuel modda dogrudan aci girisi

Simulasyon modu (ROS yokken)

Uydu Yonelim Hesaplama

Jeostasyonel uydu geometrisi

satellite_pointing.py modulu, jeostasyonel (geo-sabit) uydular icin azimuth ve elevasyon acilarini yaklasik olarak hesaplar.

Matematiksel Model:

Jeostasyonel uydu, Dünya'nin donusuyla ayni hizda dondugu icin sabit bir boylamda gorunur. Hesaplama icin basit kuresel geometri kullanilir:

```
lat = radians(latitude_deg)
delta_lon = radians(satellite_lon - observer_lon)
R_earth = 6378.137 km (Dunya yaricapi)
R_geo = 42164.0 km (jeostasyonel yorunge)
ratio = R_earth / R_geo

elevation = atan( (cos(lat)*cos(dlon) - ratio)
/ sqrt(1 - (cos(lat)*cos(dlon))^2) )
azimuth = atan2(sin(dlon), -sin(lat)*cos(dlon))
```

Desteklenen Uydular:

Uydu	Boylam	Kullanim
Turksat 4B	50.0 derece Dogu	Haberlesme, TV yayini
Turksat 5A	31.0 derece Dogu	Haberlesme, genis bant

Varsayilan konum: Ankara (39.9208 K, 32.8541 D)

Dogrulama Notu:

Bu hesap KTR (Kritik Tasarim Raporu) oncesi yazilim dogrulamasi icin yeterli yaklasimdir. Nihai sistemde WGS84, manyetik sapma ve anten montaj offsetleriyle kalibre edilmelidir.

ROS Bridge

Simulasyon ve gercek baglanti

MergenRosClient sinifi, arayuz ile ROS 2 sistemi arasinda ince bir baglanti katmani saglar.

Sinif Arayuzu:

```
class MergenRosClient:
def __init__(self):
self.connected = False

def set_mode(self, mode: str) -> str:
# ROS 2 service call veya simulasyon
return f'Simulasyon: {mode} secildi'

def send_manual_target(az, el) -> str:
# ROS 2 topic publish veya simulasyon

def save_parameters() -> str:
# ROS 2 service call veya simulasyon
```

Calisma Modlari:

Mod	Durum	Davranis	Kullanım
Simulasyon	ROS 2 yok	Tum islevler taklit Gerçek baglanti yok	Gelistirme/test bilgisayari
Gerçek	ROS 2 Humble (Ubuntu 22.04)	rclpy publisher Service client	Terminal sistemi ile entegrasyon

BOLUM 6

Simge/Elif — C++ Referans Kodu

x.cpp: İki Versiyon

Basitten gelismise PID + filtre

simge_elif/x.cpp dosyasi, Mergen stabilizasyon sistemi icin iki versiyon icerir.

Versiyon 1 (Yorum Satiri - Eski):

```
// Basit model: sabit 0.95 carpani ile filtreleme
float processIMU(float rawData) {
return rawData * 0.95f; // 'filtre'
}
// PID: integral windup korumasi YOK
float calculateCorrection(float cur, float tgt) {
integral += error; // her kademedede artar
return Kp*error + Ki*integral + Kd*derivative;
}
```

Kp=2.0, Ki=0.5, Kd=0.1 (yuksekte integral -> salinim riski)

IMU basit sabit carpan, gercek filtre degil

Windup korumasi yok -> buyuk ani hareketlerde sorun

dt parametresi yok -> sabit dongu varsayimi

Versiyon 2 (Aktif - Gelismis):

```
// Gercek low-pass filtre (alpha=0.1)
float processIMU(float rawData) {
prevFiltered = ALPHA * rawData + (1-ALPHA) * prevFiltered;
return prevFiltered;
}
```

```
// dt parametreli PID + windup korumasi  
float calculateCorrection(float cur, float tgt, float dt) {  
    if (dt <= 0.0f) return 0.0f;  
    error = tgt - cur;  
    integral = clamp(integral + error*dt, -50, 50);  
    derivative = (error - prevError) / dt;  
    return Kp*error + Ki*integral + Kd*derivative;  
}
```

Versiyon 2 Detaylari

İyi uygulama örneği

Low-Pass Filtre:

ALPHA=0.1 katsayısı ile gelen verinin sadece %10'unu alır, önceki değerin %90'unu korur. Bu, yüksek frekanslı gürültüyü bastırır.

```
prevFiltered = 0.1 * raw + 0.9 * prevFiltered
```

Küçük ALPHA = çok filtreleme (yavaş tepki)

Integral Windup Koruması:

Integral terimi +/-50 ile sınırlanmıştır. Bu sayede uzun süreli doyma durumunda integral çok büyümeyi ve sistem hedefe yaklaşıncaya kadar hızlı toparlanır.

```
integral = clamp(integral + error * dt, -50.0, 50.0);
```

dt Güvenlik Kontrolü:

dt <= 0 durumunda fonksiyon 0 döner. Bu, zamanlama hatası durumunda motorun kontrolsüz kalkmasını engeller.

```
if (dt <= 0.0f) return 0.0f;
```

Test Senaryosu:

```
Sensor girdileri: {5.0, 3.2, 1.8, 0.5, 0.1}
```

```
Her adımda: filter -> PID -> motor komutu
```

```
dt = 10ms (100 Hz kontrol)
```

BOLUM 7

13 Gereksinim Analizi

Gereksinim Karsilama Tablosu

Her sarfname maddesinin yazilim karsiligi

#	Gereksinim	Yazilim Karsiligi	Durum
1	Azimuth 0-360° surekli donus	Azimuth PID kontroloru URDF continuous joint Kablo koruma (350° uyari)	Tasarlandi
2	Elevasyon 0-90° hareket	Elevation PID kontroloru Limit kontrolu (0-90) Soft/hard limit koruma	Tasarlandi
3	Manuel mod	Python GUI ManuelCommand ROS 2 servis tasarimi Dogrudan aci girisi	Tasarlandi
4	Otomatik mod	IMU+Kalman -> Stabilizasyon SGP4 -> Uydu konumu PID -> Motor komutu QPD -> Aktif takip	Tasarlandi
5	IMU/Gyro stabilizasyon	KalmanFilter1D/EKF Quaternion estimator Stabilization controller X/Y itki mekanizmasi	Tasarlandi
6	Lazer tabanlı takip	QPD hata vektörü Aktif geri besleme PID Boresight hata kapama	Tasarlandi
7	Hedefte tekrar yonelim 8 sn	Lock state machine Spiral arama algoritmasi Zaman hedefi ve metrik	Tasarlandi
8	5 dk takip testi	Dataset simulator (5 dk) Gazebo test senaryosu CSV cikti analizi	Tasarlandi
9	GPS manuel giris	Python GUI enlem/boylam Uydu yonelim hesabi Varsayilan: Ankara	Tasarlandi

#	Gereksinim	Yazilim Karsiligi	Durum
10	Turksat 4B/5A yönelim	satellite_pointing.py Jeostasyon hesap 50.0D ve 31.0D destegi	Tasarlandı
11	Parametre kalici saklama	JSON config dosyasi ui_state.json Kalibrasyon parametreleri	Tasarlandı
12	Bilgisayar/tablet arayuzu	Tkinter tabanlı GUI 820x560 pencere ROS bridge	Tasarlandı
13	CAD-Gazebo entegrasyonu	mergen_description URDF Mesh/STL hazirligi Gazebo world dosyasi	Tasarlandı

Not: Tum gereksinimler 'Tasarlandı' seviyesindedir. CAD modeli tamamlaninca dogrulama testleri yapılacaktır.

BOLUM 8

Hiz, Dogruluk, Kaynak Kullanimi

Performans Metrikleri

Detayli sayisal degerler

Metrik	Deger	Aciklama	Not
Ana dongu periyodu	10 ms	100 Hz kontrol	Gercek zamanli
EKF predict+update	~1.2 ms	7-durumlu EKF	Dongunun %12'si
SGP4 propagate	~1.5 ms	8 iterasyon Kepler	Dongunun %15'i
PID + guvenlik	~0.2 ms	Cift eksen + monitor	Dongunun %2'si
Toplam islem suresi	~3.3 ms	5 adim toplami	%67 CPU
Telemetri	100 ms / 72 byte	10 Hz, UART	Dusuk yuk
Kalman Q (process)	0.01	Surec gurultusu	Ayar parametresi
Kalman R (olcum)	0.25	Olcum gurultusu	Ayar parametresi
Dataset ornekleme	100 Hz	5 dk = 30K satir	Dogrulama
Dataset hareket	+/-8 derece	Roll/pitch sinus	Senaryo
QPD esik	0.05	Toplam sinyal	Algila esigi
PID PWM limit	+/-1.0	Normalize	Cikis siniri

Performans Goruntuleme

Dongu suresi dagilimi

5 adimli kontrol dongusunun her bir adiminin ortalama islem suresi asagida gosterilmistir:

Adim 2: Veri Toplama	0.3ms	###
Adim 3a: EKF	1.2ms	#####
Adim 3b: SGP4	1.5ms	#####
Adim 4: PID + Guv.	0.2ms	##
Adim 5: Telemetry	0.1ms	#
TOPLAM	3.3ms	##### #####
Kalan (bos)	6.7ms	##### #####

Her dongude ~3.3ms islem + 6.7ms bos = 10ms toplam

BOLUM 9

KTR Hazirlik ve Ilerleme Plani

KTR Hazirlik Adimlari

10 adimli plan

#	Adim	Detay	Durum
1	Dataset dogrulama	Kalman/PID parametre ayari	YAPILDI
2	CAD -> Mesh export	SolidWorks -> dae/stl	Bekliyor
3	URDF/Xacro modeli	Link/joint tanımlama	Bekliyor
4	Gazebo sensor plugin	IMU, joint state, motor	Bekliyor
5	Stewart platform	Roll/pitch/yaw bozucu	Bekliyor
6	5 dk takip testi	Gazebo otomatik test	Bekliyor
7	8 sn yeniden kilitlenme	Sinyal kesilme/kilit	Bekliyor
8	Manuel/otomatik gecis	Mod testleri	Bekliyor
9	ROS -> Firmware	Teensy/STM32 port	Bekliyor
10	KTR raporu	Dataset+Gazebo+donanim	Bekliyor

Gomulu Platform Entegrasyonu:

ROS 2 kontrol node'larındaki algoritmaların gömülü platforma taşınması için izlenecek adımlar:

C++ sınıfları ROS 2 bağımlılıklarından arındırılır

HAL (Hardware Abstraction Layer) fonksiyonları yazılır

FreeRTOS görevleri: 100 Hz kontrol, 10 Hz telemetri

UART üzerinden GUI bağlantısı kurulur

On bellek (stack) ve gerçek zaman kısıtları doğrulanır

BOLUM 10

Genel Degerlendirme ve Cikarimlar

Proje Durumu

Su ana kadar yapılanlar

Kodlanmış ve Doğrulanmış Bileşenler:

- 5 adımlı gömülü yazılım (C++) - Kerim
- 7-durumlu EKF ile quaternion sensor füzyonu
- SGP4 propagator ile TLE'den uydu konumu
- Anti-windup, deadband, türev filtreli PID kontrolör
- 4 seviyeli güvenlik monitörü (SAFE/SOFT/HARD/FAULT)
- 72 byte telemetri paketi + CRC16 + komut kuyruğu
- ROS 2 Humble kontrol node'ları (C++)
- QPD lazer takip ve spiral arama algoritmaları
- Gazebo Classic simülasyon altyapısı
- Dataset tabanlı doğrulama aracı (Python)
- Tkinter GUI: manuel/otomatik, uydu seçimi, telemetri, parametre
- Teensy/STM32 firmware referans kodu
- 13/13 sartname gereksinimi 'Tasarlandı' seviyesinde

Proje Istatistikleri

Sayılarla proje

Metrik	Deger	Aciklama
Toplam dosya sayisi	~50+	Kod, konfigurasyon, dokumantasyon
Toplam kod satiri (tahmini)	~10.000+	C++ + Python
C++ baslik dosyasi	8 adet	BTK Terminal (Kerim)
ROS 2 paketi	4 adet	control, interfaces, gazebo, bringup
ROS 2 mesaj tipi	6 adet	ImuFiltered, MotorCommand, vb.
ROS 2 servis	3 adet	SetMode, SetTarget, SaveParameters
Dataset kapasitesi	30.000 satir	5 dk @ 100 Hz
Telemetri paketi	72 byte	10 Hz -> 720 byte/s
Kontrol frekansi	100 Hz	10 ms periyot
PID parametresi	14 adet	Kp, Ki, Kd vs x 2 eksen
Guvencilik seviyesi	4 seviye	SAFE -> SOFT -> HARD -> FAULT
Hata kodu	8 adet	Bitfield ile yonetim

SONUC

Genel Degerlendirme

UniS projesi, BTK Satellite Terminal Yarismasi kapsaminda Hareketli Uydu Terminali icin kapsimli bir yazilim altyapisi sunmaktadir.

Proje Yaklasimi:

CAD modeli tamamlanana kadar dataset tabanli dogrulama, CAD tamamlandiktan sonra Gazebo simulasyonu ve en son gomulu donanim entegrasyonu olmak uzere 3 asamali bir yol haritasi izlenmektedir.

Bir Sonraki Adim:

CAD modelinin tamamlanmasi ile Gazebo simulasyonunda tum sistemin dogrulanmasi ve ardindan gomulu donanim platformuna tasinmasi.

Tesekkurler

UniS Yazilim Ekibi
Kerim, Efe, Elif, Simge

Hazirlanma Tarihi: 15.05.2026